

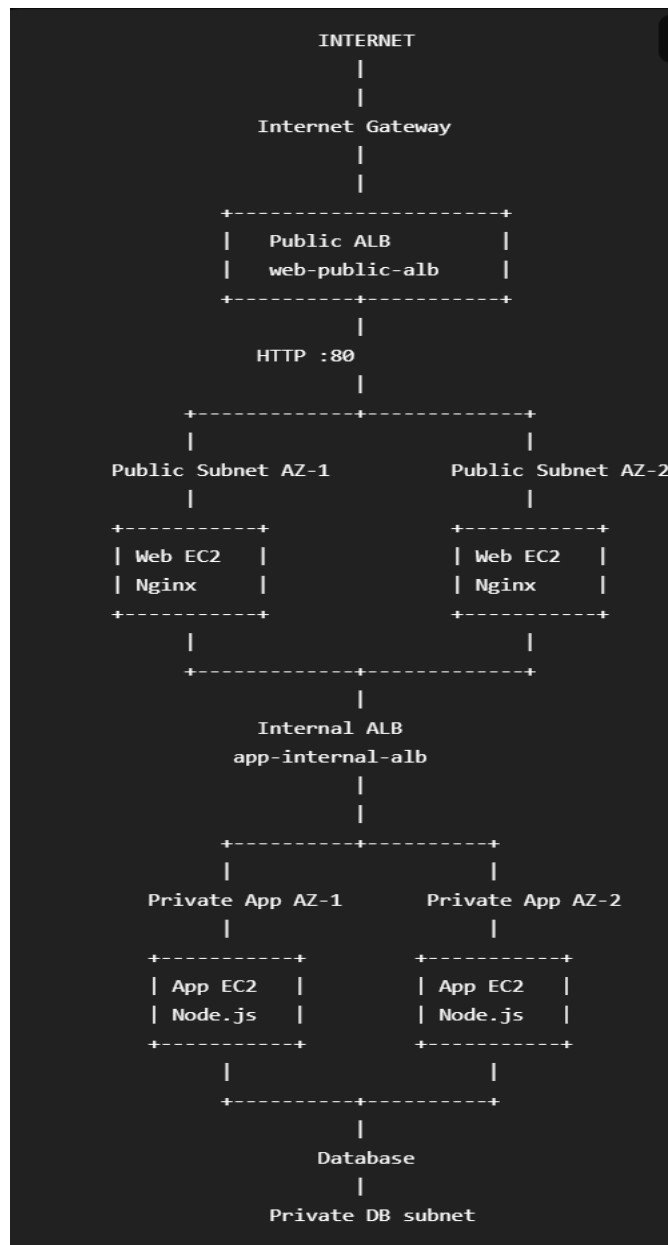
AWS 3-Tier Application

Deployment of a Scalable and Highly Available 3-Tier Application using AWS

Project Introduction

This project demonstrates the deployment of a highly available three-tier web application on Amazon Web Services (AWS). The application is divided into three layers: Web Tier, Application Tier, and Database Tier.

Architecture Diagram



VPC Documentation

Go to: **AWS Console → VPC → Your VPCs**

A dedicated VPC was created to provide an isolated networking environment for the application. The VPC contains multiple subnets distributed across Availability Zones.

VPC → Subnets

Public Subnet AZ-1

Public Subnet AZ-2

Private App Subnet AZ-1

Private App Subnet AZ-2

Private DB Subnet AZ-1

Private DB Subnet AZ-2

The screenshot displays the AWS Management Console interface for a VPC named 'three-tier-vpc' (ID: vpc-0afb08a3d6f395221). The console is in the 'Asia Pacific (Mumbai)' region, and the user is 'Pawar Krishna'.

VPC dashboard

Details

- VPC ID:** vpc-0afb08a3d6f395221
- DNS resolution:** Enabled
- Main network ACL:** acl-05f6d97df5061efd9
- IPv6 CIDR (Network border group):** -
- Encryption control ID:** -
- State:** Available
- Tenancy:** default
- Default VPC:** No
- Network Address Usage metrics:** Disabled
- Encryption control mode:** -
- Block Public Access:** Off
- DHCP option set:** dhcp-07dec94816760ae17
- IPv4 CIDR:** 10.0.0.0/16
- Route 53 Resolver DNS Firewall rule groups:** -
- DNS hostnames:** Disabled
- Main route table:** rtb-0104ab961845d5346
- IPv6 pool:** -
- Owner ID:** 771682064446

Resource map

The resource map shows the following components:

- VPC:** three-tier-vpc
- Subnets (6):**
 - ap-south-1a:** app-public-subnet, web-public-subnet, app-private-subnet
 - ap-south-1b:** web-public-subnet-2, db-private-subnet
 - ap-south-1c:** db-private-subnet-2
- Route tables (2):** rtb-0104ab961845d5346, public-route-table
- Network Connections (1):** three-tier-igw

The diagram illustrates the connectivity between these resources, showing how traffic is routed through the subnets and route tables to the internet gateway.

Database Tier

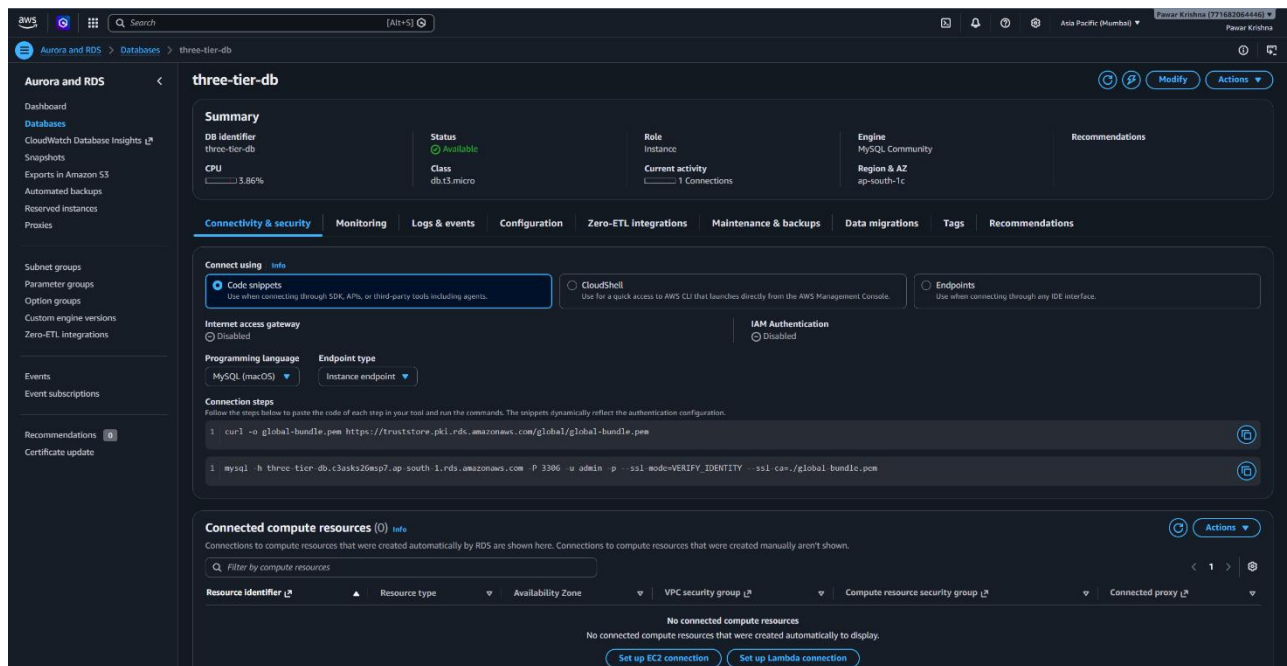
RDS MySQL

This is your **Database Tier**.

Go to: **AWS Console → RDS → Databases**

Select your database.

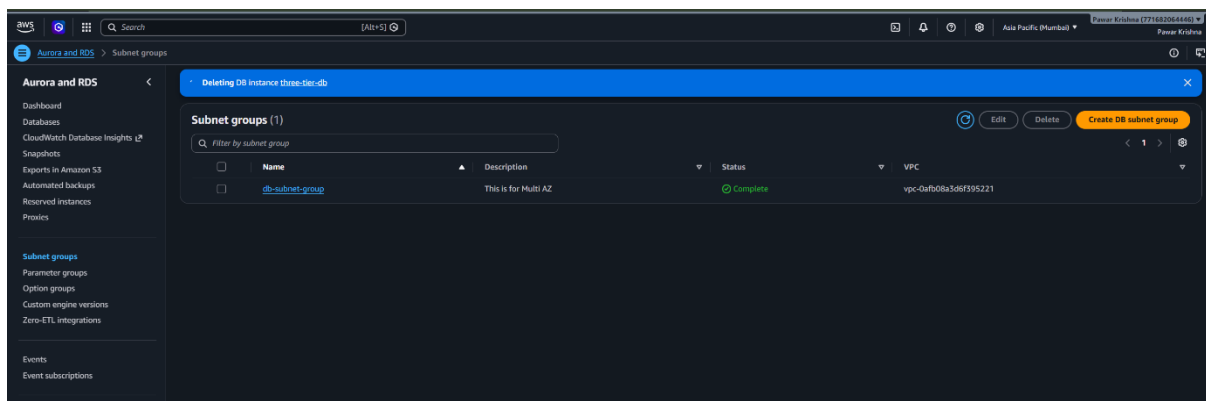
Amazon RDS for MySQL is used as the database tier. RDS manages the database infrastructure while the application communicates with MySQL through the private network.



RDS Subnet Group

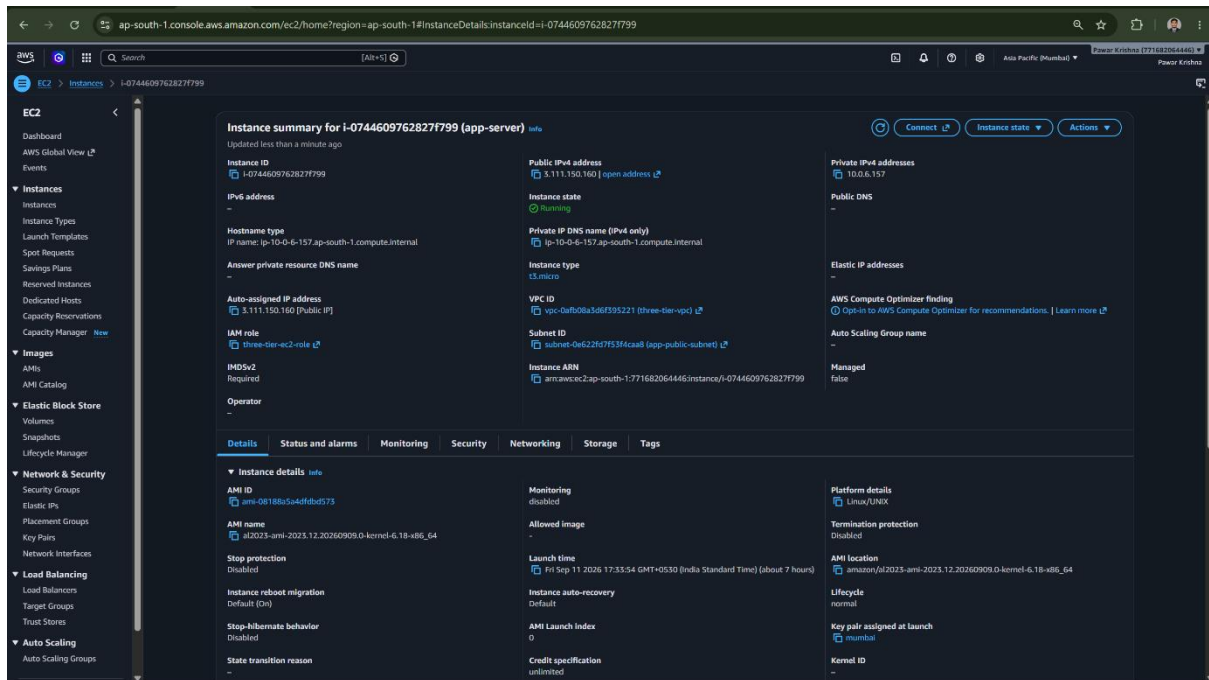
Go to: **RDS → Subnet groups**

The RDS database is deployed using a DB subnet group containing private database subnets in multiple Availability Zones.



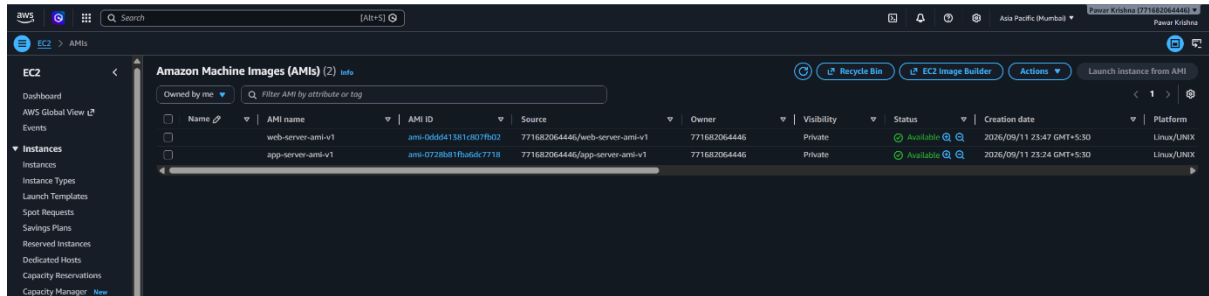
EC2 Application Server

The Application Tier runs the backend Node.js application on an Amazon EC2 instance. The backend listens on port 4000 and communicates with the RDS MySQL database.



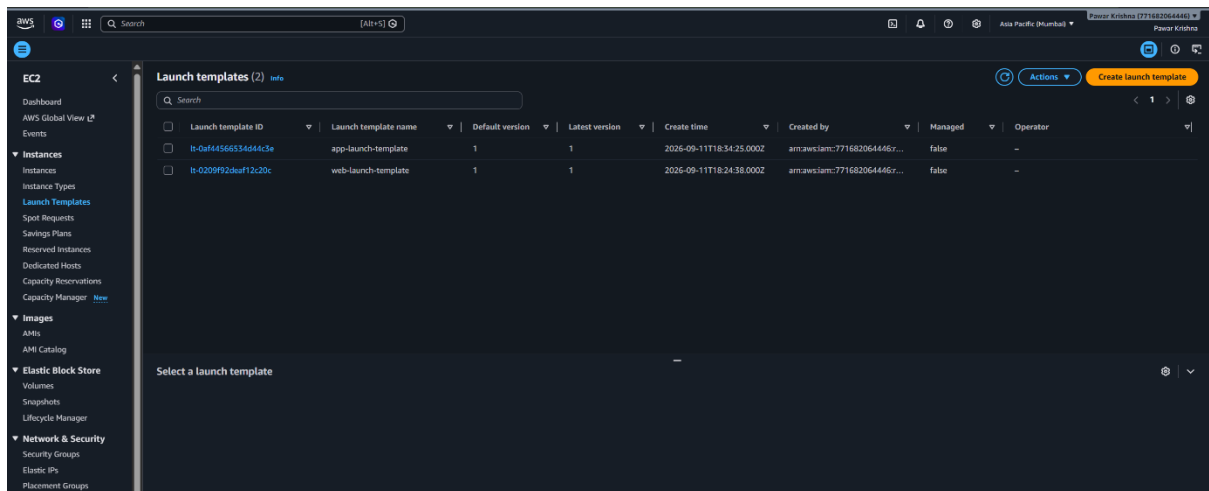
Application AMI

An Amazon Machine Image was created from the configured Application Tier EC2 instance. This AMI is used as the base image for automatically launching application servers.



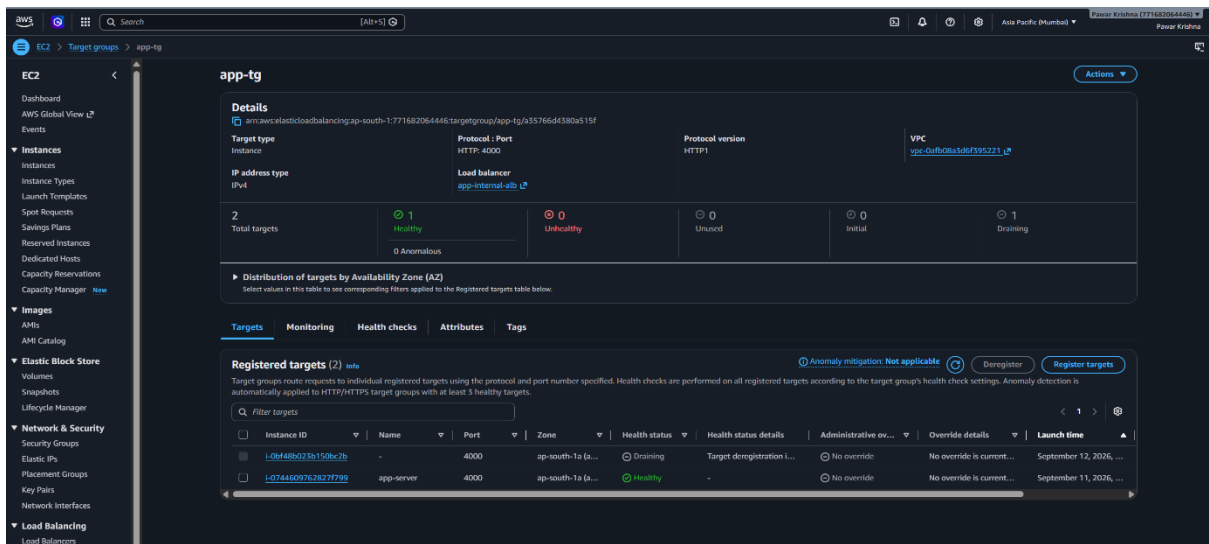
Application Launch Template

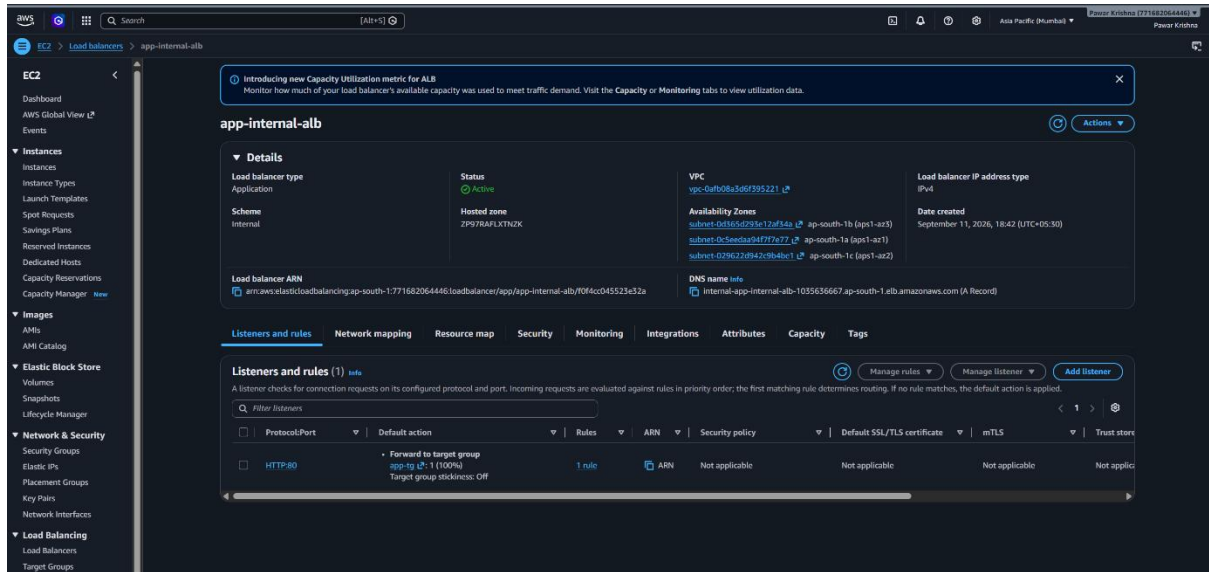
The Application Launch Template defines how new Application Tier EC2 instances should be created. It uses the Application AMI and required networking and security configuration.



Application Target Group

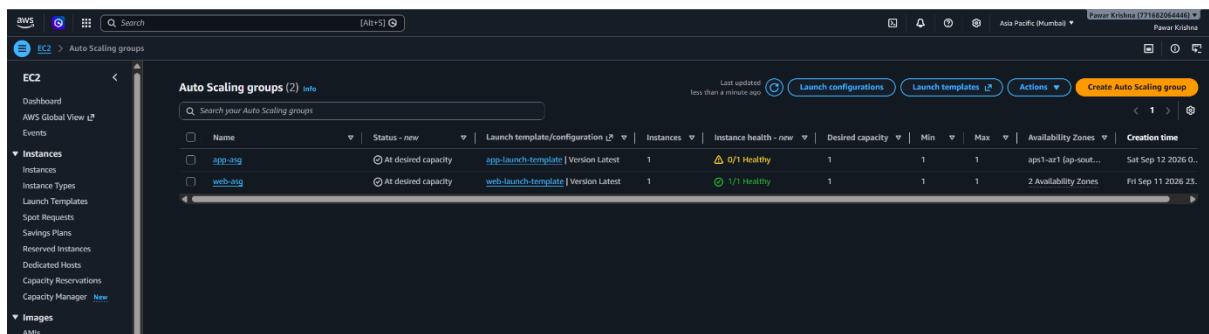
The Application Target Group contains the backend EC2 instances. It forwards traffic to port 4000 and performs health checks to verify application availability.





Application Auto Scaling Group

The Application Auto Scaling Group automatically manages Application Tier EC2 instances. It maintains the required number of instances and can launch replacement instances when required.



Web EC2 Instance

The Web Tier uses EC2 instances running NGINX. NGINX receives requests from the Public Application Load Balancer and forwards application requests to the Internal Application Load Balancer.

NGINX Configuration

```
sudo cat /etc/nginx/conf.d/app.conf
```

Also run:

```
</> Bash
```

```
sudo nginx -t
```

Expected:

```
syntax is ok
test is successful
```

Then:

```
</> Bash
```

```
sudo systemctl status nginx
```

Expected:

```
Active: active (running)
```

Write:

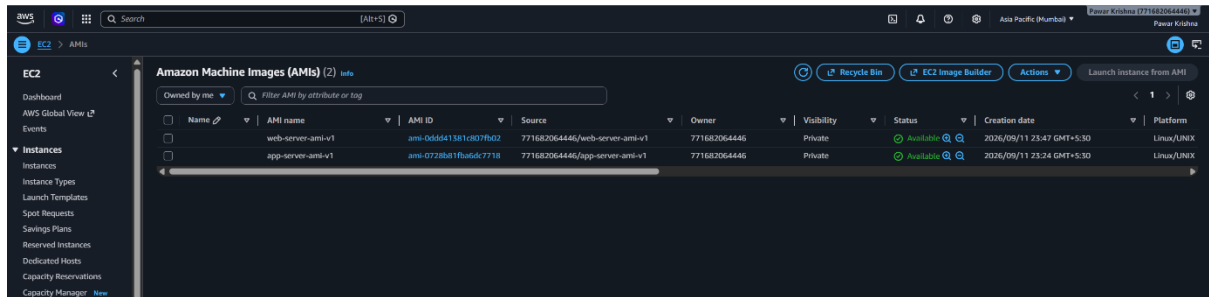
NGINX is configured as a reverse proxy in the Web Tier. Requests received by NGINX are forwarded to the Internal Application Load Balancer.

The screenshot displays the AWS Management Console interface for an EC2 instance. The left sidebar shows the navigation menu with categories like EC2, Images, Elastic Block Store, Network & Security, Load Balancing, and Auto Scaling. The main content area is titled 'Instance summary for i-0198c00ea27d65b2f (web-server)'. It provides a comprehensive overview of the instance's configuration, including its ID, state (running), IP addresses, and various settings. Below the summary, there are tabs for 'Details', 'Status and alarms', 'Monitoring', 'Security', 'Networking', 'Storage', and 'Tags'. The 'Details' tab is currently selected, showing a grid of instance details such as AMI ID, name, stop protection, and platform details.

Instance summary for i-0198c00ea27d65b2f (web-server)	
Instance ID i-0198c00ea27d65b2f	Public IPv4 address 15.206.74.92 open address
IPv6 address -	Instance state running
Instance type t3.micro	Private IP DNS name (IPv4 only) ip-10-0-1-143.ap-south-1.compute.internal
Answer private resource DNS name -	VPC ID vpc-0a8b08a3df595221 (three-tier-vpc)
Auto-assigned IP address 15.206.74.92 (Public IP)	Subnet ID subnet-0c5ceda947f7e77 (web-public-subnet)
AMI role three-tier-ec2-role	Instance ARN arn:aws:ec2:ap-south-1:771682064446:instance/i-0198c00ea27d65b2f
Operator -	Platform details Linux/UNIX
Monitoring disabled	Termination protection Disabled
Allowed image -	AMI location amazon/al2023-ami-2023.12.20260909.0-kernel-6.18-x86_64
Launch time Fri Sep 11 2026 18:57:08 GMT+0530 (India Standard Time) (about 5 hours)	Lifecycle normal
Instance auto-recovery Default	Key pair assigned at launch mumbai
AMI Launch Index 0	Kernel ID -
Credit specification unlimited	

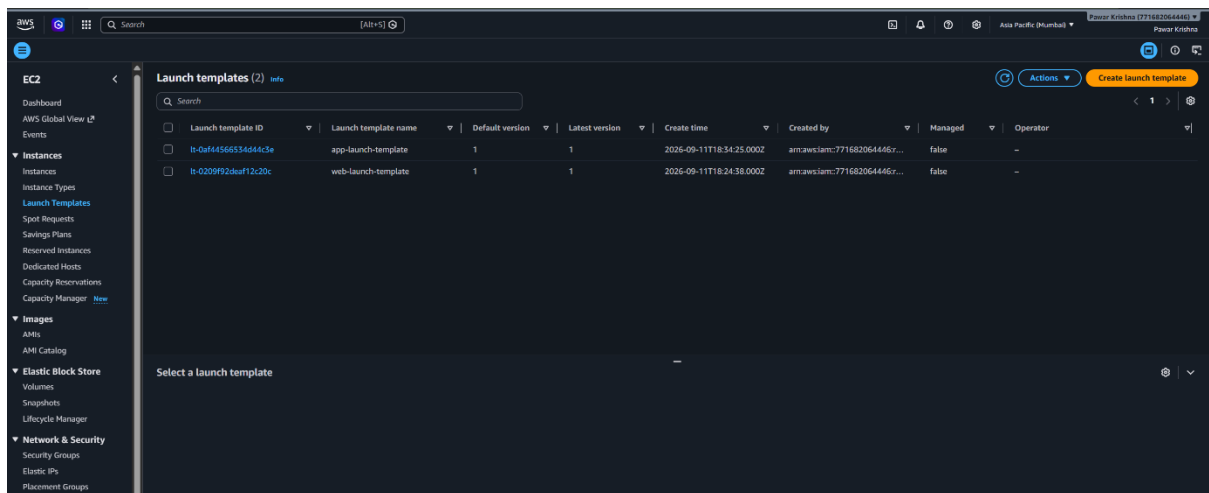
Web AMI

A Web Tier AMI was created after configuring NGINX. This AMI is used by the Web Launch Template to create Web Tier instances automatically.



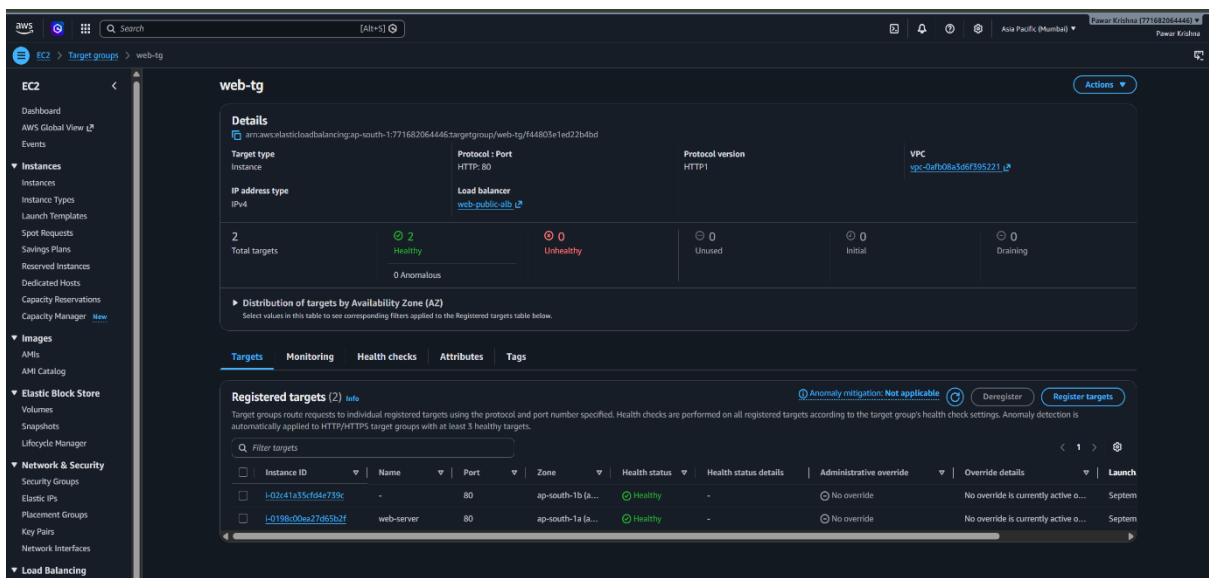
Web Launch Template

The Web Launch Template contains the configuration required to automatically launch Web Tier EC2 instances using the Web AMI.



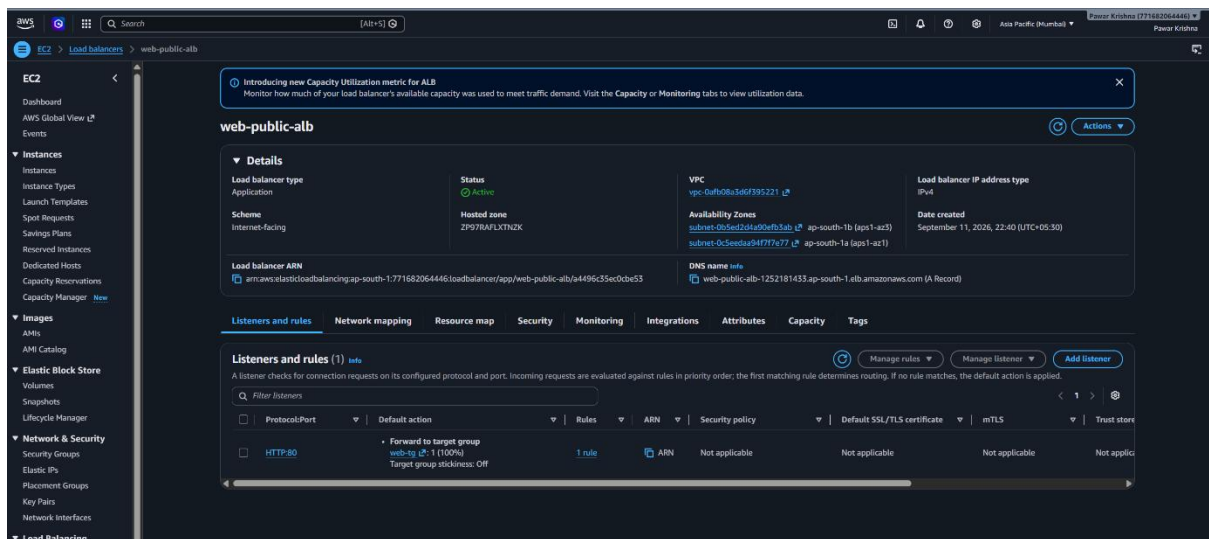
Web Target Group

The Web Target Group contains the Web Tier instances. The target group performs health checks to verify that NGINX is available and able to receive requests.



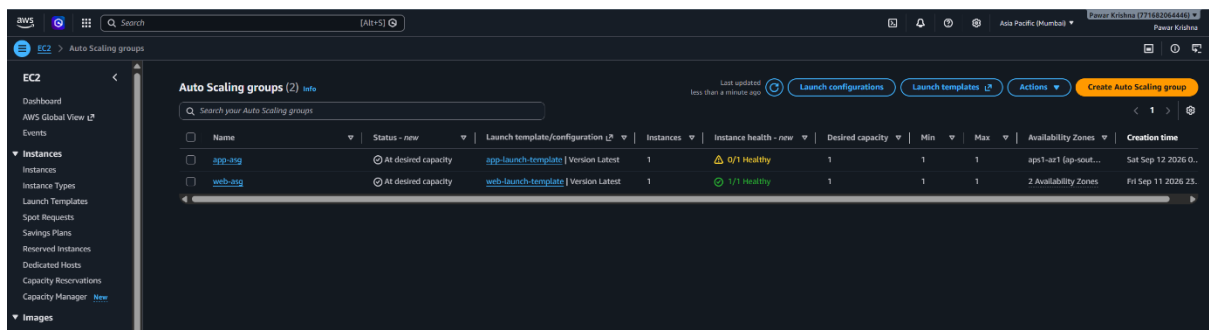
Public Application Load Balancer

The Internet-facing Application Load Balancer is the public entry point of the application. It receives HTTP requests from users and forwards them to the Web Tier.



Web Auto Scaling Group

The Web Auto Scaling Group manages the Web Tier EC2 instances. It uses the Web Launch Template to automatically create instances and maintain the desired capacity.



Final Application Testing

The final application was tested through the public Application Load Balancer. The successful root, health-check, and API requests confirm communication between the Web Tier, Application Tier, and RDS MySQL database.

Test 1 — Root

</> PowerShell



```
curl http://web-public-alb-1252181433.ap-south-1.elb.amazonaws.com/
```

Expected:

</> JSON



```
{
  "status": "success",
  "message": "Three Tier Application - App Server Working"
}
```

Test 2 — Health

</> PowerShell



```
curl http://web-public-alb-1252181433.ap-south-1.elb.amazonaws.com/api/health
```

Expected:

</> JSON



```
{
  "status": "success",
  "message": "App server and database are working"
}
```

Test 3 — Database/API

</> PowerShell



```
curl http://web-public-alb-1252181433.ap-south-1.elb.amazonaws.com/api/users
```

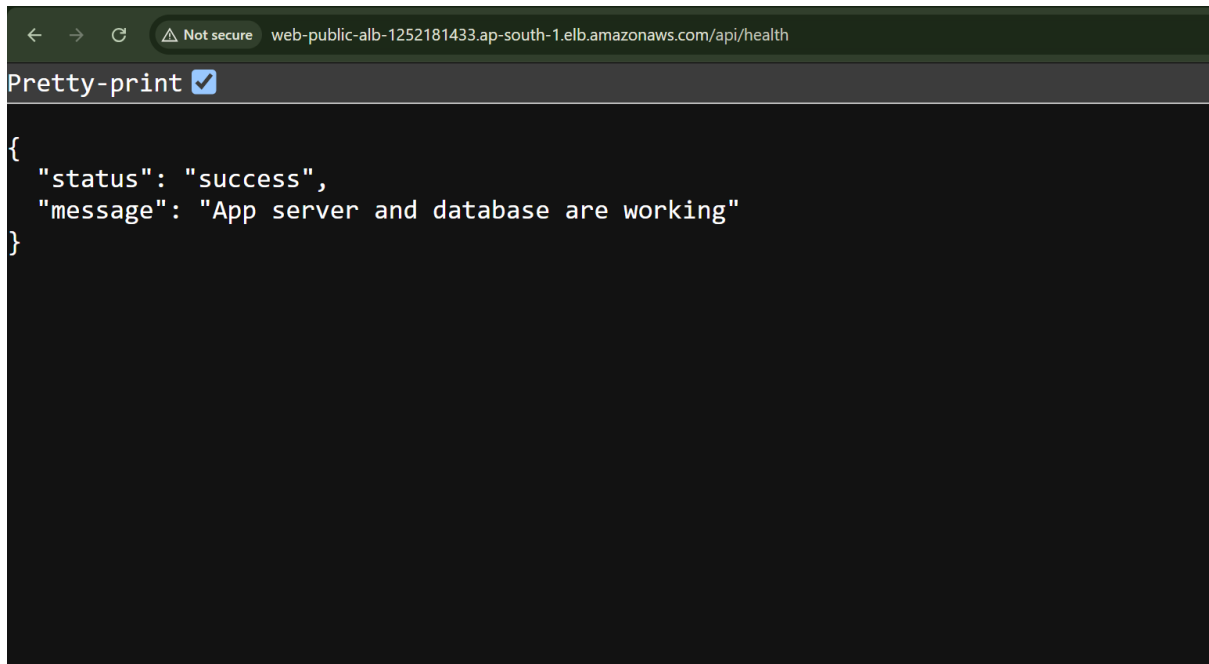
Expected:

</> JSON



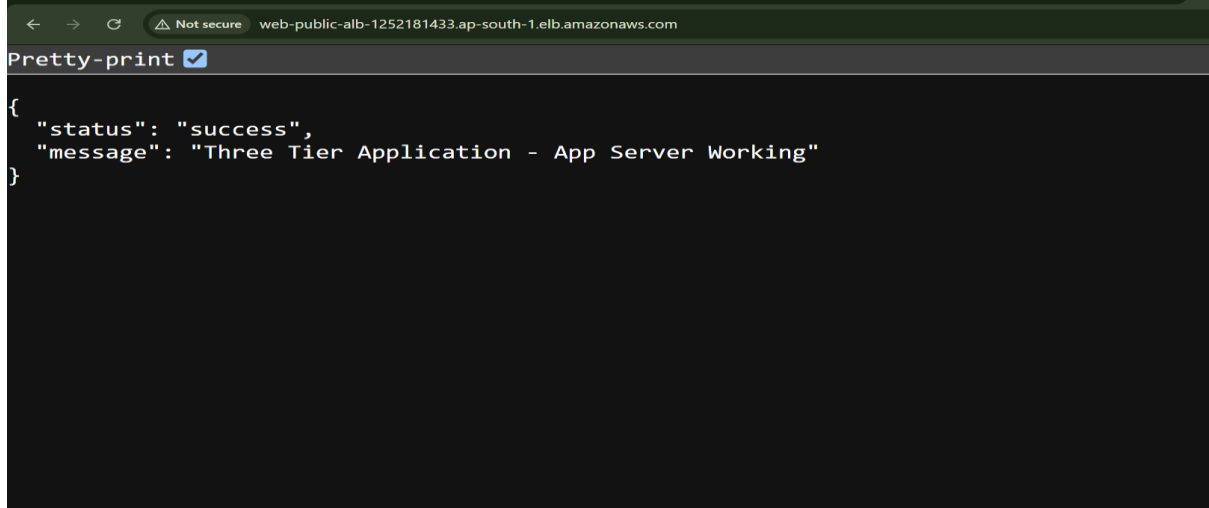
```
[
  {
    "id": 2,
    "name": "Test User",
    "email": "test@example.com"
  },
  {
    "id": 1,
    "name": "Krishna",
    "email": "krishna@example.com"
  }
]
```

Final Output



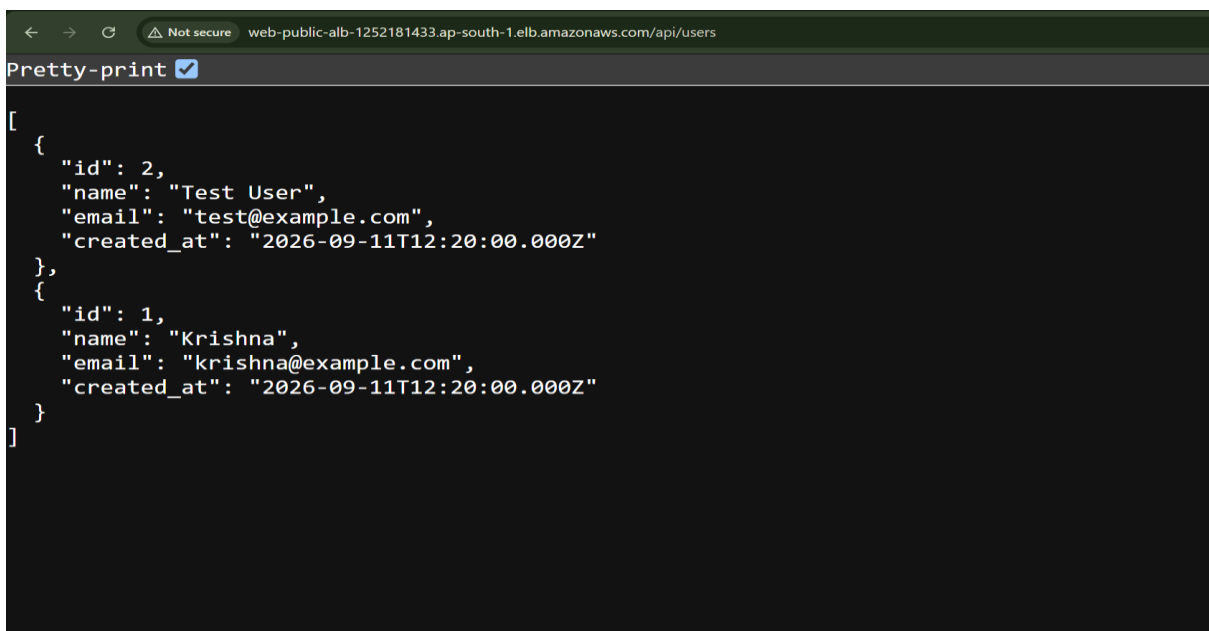
A screenshot of a web browser window. The address bar shows the URL `web-public-alb-1252181433.ap-south-1.elb.amazonaws.com/api/health` with a "Not secure" warning. The page content displays a JSON response in a dark-themed editor with a "Pretty-print" button and a checkmark icon. The JSON object indicates a successful health check.

```
{
  "status": "success",
  "message": "App server and database are working"
}
```



A screenshot of a web browser window. The address bar shows the URL `web-public-alb-1252181433.ap-south-1.elb.amazonaws.com` with a "Not secure" warning. The page content displays a JSON response in a dark-themed editor with a "Pretty-print" button and a checkmark icon. The JSON object indicates that the application is working.

```
{
  "status": "success",
  "message": "Three Tier Application - App Server Working"
}
```



A screenshot of a web browser window. The address bar shows the URL `web-public-alb-1252181433.ap-south-1.elb.amazonaws.com/api/users` with a "Not secure" warning. The page content displays a JSON array of user objects in a dark-themed editor with a "Pretty-print" button and a checkmark icon. The array contains two users: "Test User" and "Krishna".

```
[
  {
    "id": 2,
    "name": "Test User",
    "email": "test@example.com",
    "created_at": "2026-09-11T12:20:00.000Z"
  },
  {
    "id": 1,
    "name": "Krishna",
    "email": "krishna@example.com",
    "created_at": "2026-09-11T12:20:00.000Z"
  }
]
```

The three-tier web application was successfully deployed on AWS using a secure and scalable architecture. The Web Tier uses NGINX and an Internet-facing Application Load Balancer, while the Application Tier uses Node.js, an Internal Application Load Balancer, EC2, Launch Templates, AMIs, and Auto Scaling. Amazon RDS MySQL is used as the database tier. Security Groups and private subnets provide network isolation between the different tiers. Final testing through the Public Application Load Balancer successfully returned application health information and database records, confirming end-to-end communication between all three tiers.